README
for app-compatibility artifacts

❖ Contents

Content of this artifact package

The artifact folder we shared is named ***appCompatAE***, it includes two files: **artifact.zip** and **VBox.zip**. The first includes the artifacts that correspond to results presented in our paper, and it also includes reusable tools/utilities for similar future studies that we intent to share publicly. The second holds a Virtual Box VM that is used for facilitating the use/replication of our artifacts.

The ***appCompatAE*** folder is located online: https://eecs.wsu.edu/~hcai/appCompatAE/

Once the **artifact.zip** file is downloaded, unpack it. Then the following structure should be seen:

- code
  - *.sh (shell scripts)
  - *.py (python scripts)
  - installationLogs (part of our installation logs)
  - runtimeLogs (example run-time app traces and Monkey logs)
  - samples (the folder of four sample apks, for demonstrating the reusability of our utilities)
  - various other pre-generated intermediate results used for demonstration purposes and for speeding up the artifact evaluation if necessary
- data
  - benchmarks (list of apks per year)
  - *.xlsx (raw data underlying the figures / tables presented in the research paper)
  - *.docx (various study documents)
  - *.m (Matlab scripts used for data statistics and visualizations)
- README.PDF, README.DOC (i.e., this document; same content in different formats)
- Issta19-paper170.pdf (the research paper submitted and accepted)

Then, download the **VBox.zip** file, unzip it, the following files can be found therein

  - appcompatAE.vbox

- appcompat.vdi

The **code** and **data** folders include the analysis code and our dataset, respectively. Due to their large sizes, we could not include (1) all the APK files themselves, nor (2) all the app execution traces for the run-time compatibility study. Also, since our entire study took over half a year to finish, for this artifact evaluation, it is not feasible to reproduce all the original traces (for run-time compatibility study) either, nor is it feasible to reproduce all the installation logs (for the installation-time compatibility study). However, to facilitate this artifact evaluation and show the reusability of our utilities, we provided all the experimentation scripts, and gave instructions on how to use them against simple samples (see the 'Partial reproduction' section below).

The VBox zip includes the VM for VirtualBox that we created to ease the artifact evaluation, especially for exploring our study and reproducing *part of* the results presented in our research paper. Specifically, two files are included in this folder: the *appcompatAE.vbox* file is the VirtualBox configuration/project file, and the *appcompat.vdi* file is the virtual disk image that actually holds the entire VM OS along with our artifacts.

To load the VM (called *appcompatAE*), please first install VirtualBox 5.1.22 from https://www.virtualbox.org/wiki/Downloads. Then either open the *appcompatAE.vbox* file with VirtualBox or create a new Linux 64bit VM by using the VDI image *appcompat.vdi* instead of creating a new virtual hard disk. (Please make sure that your machine itself supports VM running a 64bit OS; enabling this support sometimes needs to change BIOS options.)

We have verified the workings of this VM on Ubuntu 16.04 LTS (the VM OS itself is also Ubuntu 16.04 LTS). We set 8GB RAM for this VM during our test. Once installed, launch the VM. If logon is asked (we have set auto logon on the VM), use the following credentials:

Username: **osboxes** (or an alias named **hcai**)
Password: **appcompat** (also as the sudo password)

The next two sections assume that you have launched and logged in to the VM.

Explore the artifact

First, open the terminal on the VM, then explore as follows (for convenience, you can always use "tree *dirname"* to explore a directory *dirname*'s structure; the *tree* tool has been installed).

Most of the following steps concern only browsing original inputs to the study, intermediate files produced, raw study results, and final results reported in research paper. We only provide instructions to partially reproduce the study (see next Section of this document for partial reproduction) due to the total time cost of the study. Our study needed to install 62,894 individual apps on eight different emulators (each for a different Android version); more critically, it needed to instrument 15,045 apps and run each individual app for five minutes and then to repeat this process for each of the eight emulators. The trace storage space was over 200GB.

1. Check the study tools/utilities (the 'code' folder)

The source code can be found under **/home/hcai/issta-ae/code**, where the content is the same as **code/** in *artifact.zip*. For the dynamic analysis part of our study, we used a toolkit we developed before, found at **/home/hcai/workspace/droidfax**. All dependence libraries are located at **/home/hcai/libs**. To rebuild the **droidfax** code:

cd /home/hcai/workspace/droidfax
ant clean build

Part of the droidfax (common-purpose scripts) can be found in **/home/hcai/bin** for usage convenience. We used droidfax in this study mainly for APK instrumentation that is needed for studying run-time compatibility issues.

The *code/installationLogs* folder contains part of our installation logs, generated by executing installAndroZooApks.sh on our benchmarks. Our installation-time compatibility study results came from these logs.

The *code/runtimeLogs* folder contains just a couple of example traces (and associated Monkey logs) for one year and one emulator (i.e., one SDK version). Our run-time compatibility study results came from these traces (and Monkey logs).

2. Check the study dataset (the 'data' folder)

The following table provides a mapping between the generated results and figure/table used in the research paper, along with the note on how the figure/table was produced.

Under **data**	In paper	Note
Failed Installation-iir.xlsx	Figure 2	The figure was plotted from sheet 'Summary' which summarizes the other sheets.
stacked-bar.xlsx	Figure 3	From the sheet 'install'
data-all-IIR.xlsx	Figure 4 (leftmost)	From the script drawdata_all_IIR.m on this data
installforSPSS.xlsx	Figure 4 (middle)	From the script drawdata_IIR_abi.m on this data (sheet 'abi')
installforSPSS.xlsx	Figure 4 (rightmost)	From the script drawdata_IIR_lib.m on this data (sheet 'library')
Failed Execution-rir.xlsx	Figure 5	The figure was plotted from sheet 'Summary' which summarizes the other sheets.
stacked-bar.xlsx	Figure 6	From the sheet 'run'
run-timeforSPSS.xlsx	Figure 7 (leftmost)	From the script drawdata_all_RIR.m on this data (sheet 'total (all)')
run-timeforSPSS.xlsx	Figure 7 (middle)	From the script drawdata_RIR_verify.m on this data (sheet 'verify')
run-timeforSPSS.xlsx	Figure 7 (rightmost)	From the script drawdata_RIR_native.m on this data (sheet 'native')
Installation Analysis Result.docx	Table 2	Summarized in spearman-correlation.xlsx; computed using SPSS from installforSPSS.xlsx
Run-Time Analysis	Table 3	Summarized in spearman-correlation.xlsx;

Result.docx		computed using SPSS from run-timeforSPSS.xlsx

This folder also includes two additional Word documents, "Explanations and reasons of installation failure effects.docx" and "Explanations and reasons of Run-time failure effects.docx", describing the source information we used for understanding/explaining the symptoms of installation- and run-time compatibility issues, respectively.

Reproduce the artifact (partially)

As explained above, fully reproducing all the results presented in the research paper would not be feasible with respect to the time for this artifact evaluation. However, to validate that the study toolkit actually worked to enable our full study and to show the reusability of our utilities, we prepared a way for partial reproduction.

The reproduction being partial means: reproducing our study on a smaller dataset but covering the complete study process (i.e., from prepared benchmarks to final results).

Specifically, the smaller dataset includes four apps, as randomly chosen from our entire benchmark suite. This benchmark subset is located at **samples/**.

We provide three ways of partial reproduction/replication, providing three options for the reviewers.

1. Quick reproduction
To reproduce, do

cd /home/hcai/issta-ae
bash partial-reproduce.sh

This will run our data analysis code on pre-generated installation logs and app traces (for the four apps chosen).

2. Quick demo
To run the demo, do

cd /home/hcai/issta-ae
bash demo-quick.sh

This demonstrates how our tools would be reused on a new dataset (e.g., **samples/**), but for speed, we have generated the installation logs and traces beforehand.

3. Full demo
To run the demo, do

cd /home/hcai/issta-ae
bash demo.sh

This demonstrates how our tools would be reused on a new dataset (e.g., **samples/**) from scratch, starting from the app APKs all the way to the study results. (Note, even for the very small dataset, this full demo may take 20 minutes or longer---please see 'Known issues' below for the explanation; also if this demo is executed, then the pre-generated data may be cleaned, thus the quick demo may not be executed anymore before the full demo is completed.) Please also note that, for speed, the default profiling time was set to one minute per app, although in our study the time was five minutes. Customized time period (in the unit of minutes) can be passed as the second command line argument to *runAndroZooApks_monkey.sh*. (e.g., for running each app for five minutes, use *runAndroZooApks_monkey.sh* samples 5 inside *demo.sh*).

For each of the three ways, results can be viewed from the screen, or inspected from the two files below that should be generated if the reproduction/demo executed successfully:

install_effect_sdk.txt (containing the installation-time study result)
benign_run_effect_sdk.txt (containing the run-time study result)

Please note that none of the results produced above would immediately match what is shown in the paper, because the results shown in paper are collective/aggregate (summary) statistics computed from the results demonstrated here for **all** benchmarks. The complete results that match what we presented in the paper can be found under the **data**/ folder (mostly in the Excel files).

Known issues

It is known that the Android emulator is best run on Intel-x86 architecture that has hypervisor support (kvm). However, the supervisor support is rarely available on a VM such as ours created using and running on Virtual Box.

As a makeshift, we chose to run the study in the VM on an Android AVD based on armeabi-v7a without gpu support. In consequent, the emulator runs much slower than running on a physical machine. In addition, our benchmarks may behave differently on an Intel machine (where we conducted the original study) than on the ARM machine (where we show the artifact through the VM). Thus, the performance issues and results variance can occur. However, this should only affect the partial result reproduction which involves app executions. Producing the final results from raw study results (by running */home/hcai/issta-ae/partial-reproduce.sh*) should not be affected.

Appendix: use the artifact on a different machine

Our artifacts have been tested with the following environment settings and tools/libraries (i.e., the configurations on a clean machine):

- Ubuntu 16.04 LTS
- Java 1.8.0_131 (java-1.8.0-openjdk-adm64)
- Python 2.7 (and the xlwt library)
- Tree 1.7.0 (the tree tool for browsing directories)
- Android SDK (with API 19, including tools liked adb, emulator, monkey, etc.)
- Android AVD (Nexus-One-10 with 1G SD card, 200 RAM, and intel-x86 64 system image)

- expect (year 1994 version)
- aapt (year 2014 version)
- Apktool v2.0.2
- ant v1.9.6
- vim 7.4 (optional, for editing scripts and viewing textual results)
- All the libraries as listed in /home/hcai/libs.